PhotonicsAI | AI-Driven Science and Autonomous Laboratories

Homework 1

AI, Neural Networks, Large Language Models, Agentic AI Systems

Due date: 11.10.2026

How to submit

Write the solutions to the written tasks in one PDF. That file contains the proofs, the networks, the parameter counts, and, for each code task, a description of what you built and what the run produced. Plots that belong to a result go in the same PDF.

Tasks 5 and 6 are code. Push the code for each of them to a GitHub repository. In the PDF, give the repository link and a description of what the code does, how you trained or ran it, and the numbers it printed.

Send one email to aramsargsyan@ysu.am with that PDF attached.

Task 1. Two networks a line cannot be

A language model, a classifier, and the policy inside an agent are all functions built by stacking the same piece: an affine map, then a bend. This task asks you to build two of those functions by hand, with the weights written down, before any training loop. The two targets are the absolute value and the exclusive or. A straight line cannot draw either of them.

Notation. A neuron with weight vector w, bias b, and activation σ computes

$$a = \sigma(w^\top x + b).$$

The activation you may use in a hidden unit is

$$\mathrm{ReLU}(z) = \max(0, z),$$

and the identity $\sigma(z) = z$ on a linear output. Hidden units may be reused from an earlier part. Write every weight and bias, and the activation on every unit. A diagram is enough if every arrow is labeled. Two networks that compute the same function count as the same answer. Use as few hidden units as you can, and say how many you used.

Problem.

(a) Absolute value. 6 points

Build a network with one real input x and a linear output that returns $|x|$ for every $x \in \mathbb{R}$. Use only ReLU units in the hidden layer.

Check $x \in \{-2,\ -0.5,\ 0,\ 0.5,\ 3\}$. For one negative input, write the pre-activation and the activation of each hidden unit, not only the final number.

(b) Exclusive or. 6 points

Build a network with inputs $(x_1, x_2) \in \{0,1\}^2$ and a linear output that returns the Boolean XOR. Use only ReLU units in the hidden layer.

x_1	x_2	$x_1 \oplus x_2$
0	0	0
0	1	1
1	0	1
1	1	0

Evaluate all four corners. At each corner, write the hidden activations as well as the output. A single neuron, with any weights, is not enough: say in one sentence why, by looking at the four points.

Hints, if you want them. The absolute value is two hinges joined at the origin. On the Boolean square, XOR is the absolute difference of the two inputs.

Task 2. How many parameters are in this network?

AlexNet is a stack of convolutions, two or three pooling layers, and three dense layers at the end. The network in the table has that same skeleton. The kernel sizes, strides, paddings, and widths are not AlexNet's, so the published parameter count of AlexNet is not the answer.

Problem.

The input is one RGB image, a tensor of shape $224 \times 224 \times 3$. Each convolution and each dense layer has a bias on every output channel or unit. A ReLU has no parameters. A max-pool has no parameters. There is no batch normalization and no grouped convolution: every filter reads all of the channels coming into its layer.

Padding p means p zeros on each side. For a square map of side H, a window of side k and a stride s,

$$H_{\text{out}} = \left\lfloor \frac{H + 2p - k}{s} \right\rfloor + 1.$$

The same rule gives W_{out}. A convolutional layer with C_{in} channels in, C_{out} filters, and a $k \times k$ kernel has

$$k^2 \, C_{\text{in}} \, C_{\text{out}} + C_{\text{out}}$$

parameters. A dense layer with N_{in} inputs and N_{out} units has $N_{\text{in}} N_{\text{out}} + N_{\text{out}}$ parameters. The first dense layer is fed the volume that leaves the last pool, flattened, so its N_{in} is $H \times W \times C$ for that volume.

Layer	Operation	Outputs	Kernel	Stride	Padding
C1	convolution, then ReLU	64	7×7	2	3
P1	max-pool	—	3×3	2	0
C2	convolution, then ReLU	192	5×5	1	2
P2	max-pool	—	3×3	2	0
C3	convolution, then ReLU	384	3×3	1	1
C4	convolution, then ReLU	256	3×3	1	1
C5	convolution, then ReLU	256	3×3	1	1
P3	max-pool	—	3×3	2	0
F6	dense, then ReLU	512	—	—	—
F7	dense, then ReLU	256	—	—	—
F8	dense	10	—	—	—

"Outputs" is the number of filters, or the number of units. A pool does not change the channel count. F8 is a classifier with 10 classes.

Compute the following. 10 points

(a) The output shape of every layer, written $H \times W \times C$. For a dense layer, the shape is the number of units. 4 points

(b) The number of parameters in every layer, weights and biases together. 4 points

(c) The total number of parameters in the network. 2 points

Task 3. Softmax is a probability mass function

A classifier has K classes, a finite set $\{1, \ldots, K\}$. Softmax turns a vector of real scores $z = (z_1, \ldots, z_K)$ into one number per class:

$$s_i(z) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}, \qquad i = 1, \ldots, K.$$

A probability mass function on that finite set is a vector p with $p_i \geq 0$ and $\sum_i p_i = 1$. A probability density is a different object: a nonnegative function of a continuous outcome, whose integral over that outcome is 1.

Problem.

Prove that $s(z)$ is a probability mass function on the K classes, and that it is not a probability density. 6 points

(a) Prove $s_i(z) > 0$ for every class i, and prove $\sum_{i=1}^{K} s_i(z) = 1$, for every real score vector z. These two facts are the definition of a mass function on $\{1, \ldots, K\}$. Say that in one sentence at the end of the proof. 3 points

(b) Using the sum you wrote in (a), explain why s is not a density. The normalization in (a) runs over the class index $j = 1, \ldots, K$, and the output is one number for each class. A density is a function of a continuous outcome, normalized by an integral. Softmax has no continuous outcome in it. 3 points

Task 4. A dot product does not know the seat

Unmasked attention decides how much token i listens to token j by a single number, the dot product of the query q_i with the key k_j:

$$(QK^\top)_{ij} = q_i^\top k_j.$$

A dot product takes two vectors and returns a scalar. The integers i and j, the seats, do not appear in that formula. Carry the same two tokens into different seats and $q^\top k$ is the same number. What changes is only which row and which column of the score matrix that number occupies. That is the sense in which attention is position-agnostic, and it is already true at the dot product, before the softmax and before the values are mixed.

Problem.

Rows of $X \in \mathbb{R}^{n \times d}$ are the token vectors. With projection matrices W_Q, W_K, and W_V,

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V,$$

$$\mathrm{Att}(X) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

Here d_k is the dimension of each key vector. Softmax acts on each row separately. There is no attention mask. Let π be a permutation of $\{1, \ldots, n\}$, and let P be the permutation matrix with $P_{i,\pi(i)} = 1$ and zeros elsewhere. The i-th row of PX is the $\pi(i)$-th row of X. You may use $P^\top P = I$, and the fact that the softmax of a reordered vector is the reordered softmax.

Prove the following. 8 points

(a) The score is a dot product. 3 points

Write $(QK^\top)_{ij} = q_i^\top k_j$. If the rows of X are reordered by π, the new query in seat i is the old query of the token that moved there, and the same is true of the keys. Show

$$(q'_i)^\top k'_j = q_{\pi(i)}^\top k_{\pi(j)}.$$

In one sentence, follow one token that moves from seat 1 to seat 3: the dot products of its vector with the other token vectors keep their values, and what changes is the seat those values occupy.

Then compute it on two tokens. Let the rows be $u = (1, 0)$ and $v = (0, 1)$, and take $W_Q = W_K = I$, so the scores are the dot products of the rows themselves. Write the 2×2 table of dot products with u in the first seat. Swap the seats and write the table again. The four numbers are the same. Say which entries moved.

(b) The whole layer follows the tokens. 3 points

Use (a) to prove $\mathrm{Att}(PX) = P\,\mathrm{Att}(X)$. Because every score is the dot product in (a), the score matrix of the reordered tokens is $P(QK^\top)P^\top$. Row-wise softmax follows that same reordering, and so does the mixture with the values. In one sentence, say what this means for the output row that belongs to a given token.

On the same two tokens, with $W_V = I$ and with the factor $1/\sqrt{d_k}$ dropped, compute $\mathrm{Att}(X)$ and $\mathrm{Att}(PX)$.

(c) Position has to enter before the dot product. 2 points

Positional encodings $e_1, \ldots, e_n$ are tied to the seat. They are added to the token vectors, and only then does the dot product run, so the score of seats i and j becomes $(x_i + e_i)^\top (x_j + e_j)$. The model therefore receives $X + E$. After the tokens are reordered and the encodings stay with the seats, the model receives $PX + E$. Show that if $e_1 \neq e_2$, then for a swap of the two seats $PX + E \neq P(X + E)$. The two sides differ in the vectors that enter the dot product. A stack of attention and of layers that use the same function on every row still satisfies $F(PX) = PF(X)$. Order is visible because the encodings changed the vectors before any dot product was taken.

Task 5. PINN (Physics Informed Neural Network) for the Silverbox oscillator

The Silverbox is a laboratory electronic circuit built to act as a Duffing oscillator: a second-order system with a spring that stiffens as the displacement grows. The same equation shows up for a mass on a cubic spring, and the circuit was built so that the equation can be measured cleanly. The public record is the Silverbox benchmark of Wigren and Schoukens (European Control Conference,

2013), file `SNLS80mV.mat`, available from https://www.nonlinearbenchmark.org/. Channel `V1` is the input voltage $u(t)$. Channel `V2` is the output voltage $y(t)$. Both are sampled at

$$f_s = 610.35 \text{ Hz}, \qquad \Delta t = 1/f_s.$$

The file contains 131,072 samples. The estimation experiment is a random-phase multisine, and it begins at sample 40650 (indices into the file, starting at 0).

Problem.

The excerpt. Use the 4096 samples starting at index 42650, that is the slice [42650 : 46746]. This sits 2000 samples after the multisine begins. Subtract the mean of u on this excerpt and the mean of y on this excerpt, and use those centered signals afterwards. The first 3072 samples are the training record, about 5.03 s. The last 1024 samples are the test record, about 1.68 s. On the test record the input u is known, because it is the voltage that was applied. The output y is what the model has to produce.

The differential equation. Write the centered output as y and the centered input as u. The circuit is modeled by

$$\ddot{y} + a\,\dot{y} + b\,y + c\,y^3 = g\,u(t). \tag{1}$$

The four coefficients are properties of the circuit.

- a multiplies the velocity. It is the damping. On this circuit it is a few tens of s^{-1}.
- b multiplies y. It is the linear stiffness, in s^{-2}. The output of this excerpt is dominated by frequencies near 70 Hz.
- c multiplies y^3. It is the cubic stiffness, the term a linear spring does not have. At the peaks of this excerpt, where $|y|$ is near 0.2 V, the cubic force is a visible fraction of the linear force.
- g multiplies the input voltage. It is an input gain, because u and y are voltages on opposite sides of the circuit.

They are not the weights of the network. The acceleration is produced by a network that has to represent this force.

The network. The acceleration is a multilayer fully connected network, implemented in PyTorch. You choose the depth, the widths, the activation, and how the inputs and the output are scaled. With velocity $v = \dot{y}$, the state is the pair (y, v) and

$$\dot{y} = v, \qquad \dot{v} = \alpha_\theta(y, v, u). \tag{2}$$

Integrate that state at step Δt, driven by the measured input. Classical Runge–Kutta of order 4 is a sound choice for the step. The excerpt lasts 6.7 s near 70 Hz, several hundred cycles, so the integrator is what draws the waveform. Try more than one architecture, and for each architecture tune the training hyperparameters: the optimizer, the learning rate, the number of steps, and the length of the rollout you train on. Keep the version with the best test score under the metric below.

The metric. After the weights of an architecture are fixed, do one free run of the whole excerpt. Start at the first sample, with the measured position and a finite-difference velocity, and integrate through the training samples and on into the test samples. Keep the same state when the test segment begins. On the test segment the rollout may read u, and it may not read y. The performance of the architecture is the root-mean-square error of this free run on the 1024 test samples,

$$\text{RMSE} = \sqrt{\frac{1}{1024}\sum_{i=1}^{1024}\left(\hat{y}_i - y_i\right)^2}.$$

A smaller test RMSE is a better version. Predicting zero scores the RMS of the test output, about 0.06 V, so that number is the baseline. Also compute the same free-run RMSE on the 3072 training samples. A network that fits the training record and then drifts on the test segment shows up as a large gap between the two.

In the solution PDF, describe the architecture you kept and the hyperparameters you settled on, the state equation, and how you trained it. Say which other settings you tried and what test RMSE each one got. Give both free-run RMSE numbers for the kept version, and a plot of measured against predicted output on the test segment. The prediction in that plot is the free run above. Push the code to a GitHub repository and put the link in the PDF. 12 points

Task 6. Agentic System

Task 5 asks for the differential equation, a plan for the network, and a script that has to be written and run. This task builds a system of four roles that splits that work, and then asks the system to do Task 5.

Problem.

The user speaks only to the orchestrator. Every other role speaks only when an edge of the graph gives it the turn. The dialogue is the state of the graph. Each turn carries the name of the role that produced it and the text of what that role said.

The graph. Build the system with LangGraph, as a `StateGraph`. The orchestrator, the scientist, the general agent, and the executor are four nodes. The state they share is the dialogue. An edge from one node to another hands that agent the turn: the node reads the messages already in the state, including what the other agents wrote, and appends its own message. The orchestrator node chooses the next edge, to the scientist, the general agent, the executor, or the end. After a specialist node runs, an edge returns the turn, so the next speaker sees the new message. That is how the agents communicate: each one writes into the shared state, and the next one reads it.

Orchestrator. On each turn the orchestrator sees the dialogue and returns a decision with three fields: the next role, the instruction that role should follow, and a one-sentence reason. The next role is one of `scientist`, `general`, `executor`, or `finish`. That decision is itself a call to a model, which reads the dialogue and chooses. A table of keywords, or a fixed script that always calls the same roles in the same order, does not count. The orchestrator writes no theory and no code, and it has no coding tool. It stops on `finish`, or after 8 decisions, whichever comes first. It may finish only after a turn from the executor contains a test RMSE printed by a script that actually ran.

Scientist. This role owns the theory and the plan. From the problem it writes the differential equation, the network it proposes, the state equation to integrate, how to train, the free-run test, and a numbered list of steps that is specific enough to code. It has no coding tool. It does not claim that a number was measured unless an executor turn already printed that number.

General agent. This role answers a high-level question that sits beside the fit: what a record in the file is, what a term in the problem means, what a reported error is measuring. It has no coding tool, and it does not write the fitting plan. When a question is the plan for the differential equation, the orchestrator sends it to the scientist. When a question is a traceback, the orchestrator sends it to the executor.

Executor and the coding tool. The executor is the only role that receives the coding tool. The tool does two things, and both are required.

- Write a Python file into a working directory chosen for this run.

- Run that file, with a time limit of five minutes, and return the exit code, the standard output, and the standard error.

The executor implements the latest plan from the scientist, in a new file. It returns the free-run RMSE the script printed. When the script fails, it returns the traceback. A later executor turn may repair the script. The scientific plan stays the one the scientist wrote: the same differential equation and the same free-run test.

Validate the system on Task 5. Give the system Task 5 and let it try to solve that task. In the solution PDF, show the process and the result: which agent did what, and the free-run error the run printed. Push the code to a GitHub repository and put the link in the PDF. 14 points